



WebGL e la Pipeline Grafica: Gestione dei Dati e Conversione Raster

In questa fase passiamo dalla grafica 2D del Canvas alla potenza di calcolo della GPU. WebGL non disegna semplici oggetti, ma elabora flussi di vertici per colorare pixel tramite la pipeline programmabile.

Indice

- [WebGL e la Pipeline Grafica: Gestione dei Dati e Conversione Raster](#)
 - [Indice](#)
 - [1. WebGL e l'Architettura degli Shader](#)
 - [2. Clipping: Determinare la Visibilità](#)
 - [Clipping di Linee \(Algoritmo di Cohen-Sutherland\)](#)
 - [Clipping di Poligoni \(Sutherland-Hodgman\)](#)
 - [3. Rasterizzazione: Conversione in Pixel](#)
 - [Linee: L'algoritmo di Bresenham](#)
 - [Poligoni: Scan-Line Fill](#)
 - [4. Problematiche del Campionamento](#)
 - [5. Esempio Pratico e Flusso WebGL](#)
-

1. WebGL e l'Architettura degli Shader

WebGL è l'interfaccia che permette a JavaScript di comunicare con la scheda video. Il cuore di questo sistema sono i programmi che girano direttamente sulla GPU, chiamati **Shaders**, scritti in linguaggio **GLSL**.

- **Vertex Shader:** Riceve i vertici dei modelli 3D. Il suo compito è calcolare la posizione finale del vertice nello spazio di ritaglio (Clip Space) e preparare i dati (colori, normali) per la fase successiva.
- **Fragment Shader:** Riceve i "frammenti" (pixel potenziali) generati dalla rasterizzazione. Per ogni pixel, calcola il colore finale basandosi su luci, materiali e texture.

- **VBO (Vertex Buffer Objects):** Sono contenitori di memoria sulla GPU dove carichiamo le coordinate dei vertici per minimizzare lo scambio di dati tra CPU e scheda video durante il disegno.
-

2. Clipping: Determinare la Visibilità

Prima di trasformare la geometria in pixel, il sistema deve scartare tutto ciò che si trova fuori dall'inquadratura per ottimizzare le prestazioni.

Clipping di Linee (Algoritmo di Cohen-Sutherland)

Divide lo spazio attorno alla finestra di visualizzazione in 9 regioni, assegnando a ciascuna un codice a 4 bit (**outcode**).

- **Test dei bit:** Se entrambi i punti di una linea hanno codice `0000`, la linea è dentro. Se l'operazione di AND logico tra i codici degli estremi non è zero, la linea è sicuramente fuori.
- **Intersezione:** Se la linea attraversa i bordi, l'algoritmo calcola il punto di intersezione con il confine della finestra e "taglia" la parte esterna, ripetendo il test sul segmento rimasto.

Clipping di Poligoni (Sutherland-Hodgman)

Tagliare un poligono richiede di mantenere la figura chiusa. L'algoritmo confronta il poligono contro ogni bordo della finestra (Sinistra, Destra, Alto, Basso) in sequenza. Per ogni coppia di vertici, decide se mantenere il vertice corrente, scartarlo o generare un nuovo punto di intersezione sul bordo, creando così una nuova lista di vertici.

3. Rasterizzazione: Conversione in Pixel

Una volta pulita la geometria, bisogna trasformare i vettori matematici in una griglia di pixel (raster).

Linee: L'algoritmo di Bresenham

È lo standard industriale per disegnare segmenti. Evita calcoli pesanti (divisioni o numeri decimali) usando solo aritmetica intera. Calcola un "errore" accumulato passo dopo passo per decidere se il pixel successivo deve restare sulla stessa riga o spostarsi in diagonale per seguire la pendenza ideale della linea.

Poligoni: Scan-Line Fill

Per riempire l'interno di un poligono:

1. Il sistema traccia linee orizzontali virtuali (**scan-lines**) che attraversano il poligono.
 2. Trova i punti di intersezione con i lati e li ordina da sinistra a destra.
 3. Colora i pixel compresi tra le coppie di intersezioni seguendo la regola di riempimento.
-

4. Problematiche del Campionamento

Il passaggio dal continuo (geometria) al discreto (pixel) introduce artefatti:

- **Aliasing:** Le classiche "seghettature" sui bordi diagonali. Si risolve con tecniche di **Antialiasing** (come il Multisampling) che sfumano i bordi per ingannare l'occhio.
 - **Z-Buffer:** Durante la rasterizzazione, il sistema consulta il *Depth Buffer* per ogni pixel. Se il nuovo frammento è più lontano di quello già memorizzato, viene scartato. Questo garantisce che gli oggetti coperti da altri non vengano disegnati sopra.
-

5. Esempio Pratico e Flusso WebGL

Il processo mostrato nelle slide per far funzionare WebGL segue questi step:

1. **Inizializzazione:** Ottenere il contesto `webgl` dal canvas.
2. **Shaders:** Caricare, compilare e linkare Vertex e Fragment Shader in un "Program".
3. **Buffer:** Creare i buffer sulla GPU e caricarvi i dati dei vertici.
4. **Attributes:** Collegare le variabili dello shader ai dati nei buffer.

5. **Draw:** Lanciare il comando di disegno (`gl.drawArrays`), attivando l'intera pipeline hardware.